

1. Write a program to implement CRC for Error Detection.**Program:**

```
def crc(data, divisor):  
    n = len(divisor)  
    temp = list(data + '0' * (n - 1))  
  
    # XOR division  
    for i in range(len(data)):  
        if temp[i] == '1':  
            for j in range(n):  
                temp[i + j] = '0' if temp[i + j] == divisor[j] else '1'  
  
    remainder = ".join(temp[len(data):])  
    return data + remainder  
  
def verify(received, divisor):  
    n = len(divisor)  
    temp = list(received)  
  
    # Run the same XOR division over the received message  
    for i in range(len(received) - n + 1):  
        if temp[i] == '1':  
            for j in range(n):  
                temp[i + j] = '0' if temp[i + j] == divisor[j] else '1'  
  
    # If any remainder bit is '1', an error occurred  
    remainder = ".join(temp[-(n - 1):])
```

```
    return '1' in remainder
```

```
# --- SENDER SIDE ---
```

```
data = input("Enter data bits: ")
```

```
divisor = input("Enter divisor bits: ")
```

```
codeword = crc(data, divisor)
```

```
print("CRC Codeword to send:", codeword)
```

```
print("-" * 30)
```

```
# --- RECEIVER SIDE ---
```

```
received = input("Enter received codeword: ")
```

```
if verify(received, divisor):
```

```
    print("Error detected in transmission!")
```

```
else:
```

```
    print("No error detected. Data received successfully.")
```

Output:

```
Enter data bits: 10000101
```

```
Enter divisor bits: 1001
```

```
CRC Codeword to send: 10000101111
```

```
-----
```

```
Enter received codeword: 10000101111
```

```
No error detected. Data received successfully.
```

2. Write a program to implement Checksum for Error Detection.**Program:**

Function to find the Checksum of Sent Message

```
def findChecksum(SentMessage, k):
```

```
    # Dividing sent message in packets of k bits.
```

```
    c1 = SentMessage[0:k]
```

```
    c2 = SentMessage[k:2*k]
```

```
    c3 = SentMessage[2*k:3*k]
```

```
    c4 = SentMessage[3*k:4*k]
```

```
    # Calculating the binary sum of packets
```

```
    Sum = bin(int(c1, 2)+int(c2, 2)+int(c3, 2)+int(c4, 2))[2:]
```

```
    # Adding the overflow bits
```

```
    if(len(Sum) > k):
```

```
        x = len(Sum)-k
```

```
        Sum = bin(int(Sum[0:x], 2)+int(Sum[x:], 2))[2:]
```

```
    if(len(Sum) < k):
```

```
        Sum = '0'*(k-len(Sum))+Sum
```

```
    # Calculating the complement of sum
```

```
    Checksum = "
```

```
    for i in Sum:
```

```
        if(i == '1'):
```

```
            Checksum += '0'
```

```
        else:
```

```
        Checksum += '1'
    return Checksum

# Function to find the Complement of binary addition of
# k bit packets of the Received Message + Checksum
def checkReceiverChecksum(ReceivedMessage, k, Checksum):

    # Dividing sent message in packets of k bits.
    c1 = ReceivedMessage[0:k]
    c2 = ReceivedMessage[k:2*k]
    c3 = ReceivedMessage[2*k:3*k]
    c4 = ReceivedMessage[3*k:4*k]

    # Calculating the binary sum of packets + checksum
    ReceiverSum = bin(int(c1, 2)+int(c2, 2)+int(Checksum, 2) +
                      int(c3, 2)+int(c4, 2)+int(Checksum, 2))[2:]

    # Adding the overflow bits
    if(len(ReceiverSum) > k):
        x = len(ReceiverSum)-k
        ReceiverSum = bin(int(ReceiverSum[0:x], 2)+int(ReceiverSum[x:], 2))[2:]

    # Calculating the complement of sum
    ReceiverChecksum = ""
    for i in ReceiverSum:
        if(i == '1'):
            ReceiverChecksum += '0'
```

else:

ReceiverChecksum += '1'

return ReceiverChecksum

Driver Code

SentMessage = input("Enter the Sent Message: ")

k = 8

#ReceivedMessage = "10000101011000111001010011101101"

ReceivedMessage = SentMessage

Calling the findChecksum() function

Checksum = findChecksum(SentMessage, k)

Calling the checkReceiverChecksum() function

ReceiverChecksum = checkReceiverChecksum(ReceivedMessage, k,
Checksum)

Printing Checksum

print("SENDER SIDE CHECKSUM: ", Checksum)

print("RECEIVER SIDE CHECKSUM: ", ReceiverChecksum)

finalsum=bin(int(Checksum,2)+int(ReceiverChecksum,2))[2:]

Finding the sum of checksum and received checksum

finalcomp=""

for i in finalsum:

if(i == '1'):

finalcomp += '0'

else:

finalcomp += '1'

```
# If sum = 0, No error is detected
if(int(finalcomp,2) == 0):
    print("Receiver Checksum is equal to 0. Therefore,")
    print("STATUS: ACCEPTED")

# Otherwise, Error is detected
else:
    print("Receiver Checksum is not equal to 0. Therefore,")
    print("STATUS: ERROR DETECTED")
```

Output:

```
Enter the Sent Message: 10001000100010001000100010001000
SENDER SIDE CHECKSUM: 11011101
RECEIVER SIDE CHECKSUM: 00100010
Receiver Checksum is equal to 0. Therefore,
STATUS: ACCEPTED
```

3. Write a program to implement Hamming Code for Error Detection and Correction.**Program:**

```
# Hamming Code
```

```
data = input("Enter 4-bit data: ")
```

```
# Positions: 1 2 3 4 5 6 7
```

```
#      P1 P2 D1 P4 D2 D3 D4
```

```
d1, d2, d3, d4 = map(int, data)
```

```
p1 = d1 ^ d2 ^ d4
```

```
p2 = d1 ^ d3 ^ d4
```

```
p4 = d2 ^ d3 ^ d4
```

```
code = [p1, p2, d1, p4, d2, d3, d4]
```

```
print("Hamming Code:", ".join(map(str, code)))
```

```
# Receiver
```

```
received = input("Enter received code: ")
```

```
r = list(map(int, received))
```

```
s1 = r[0] ^ r[2] ^ r[4] ^ r[6]
```

```
s2 = r[1] ^ r[2] ^ r[5] ^ r[6]
```

```
s4 = r[3] ^ r[4] ^ r[5] ^ r[6]
```

```
error = s4 * 4 + s2 * 2 + s1
```

```
if error == 0:
```

```
    print("No Error")
```

```
else:
```

```
    print("Error at position:", error)
```

```
    r[error - 1] ^= 1
```

```
    print("Corrected Code:", ".join(map(str, r)))
```

Output:

```
Enter 4-bit data: 1011
Hamming Code: 0110011
Enter received code: 0110111
Error at position: 5
Corrected Code: 0110011
```

4. Write a program to implement Byte stuffing and Bit stuffing.

Bit Stuffing Program:

```
data = input("Enter binary data: ")
```

```
count = 0
```

```
stuffed = ""
```

```
for bit in data:
```

```
    stuffed += bit
```

```
    if bit == '1':
```

```
        count += 1
```

```
        if count == 5:
```

```
            stuffed += '0'
```

```
            count = 0
```

```
    else:
```

```
        count = 0
```

```
print("After Bit Stuffing:", stuffed)
```

Output:

```
Enter binary data: 1111111111
After Bit Stuffing: 111110111110
```

Byte Stuffing Program:

```
data = input("Enter data: ")
```

```
flag = 'FLAG'
```

```
escape = 'ESCAPE'
```

```
stuffed = flag
```

```
for ch in data:
```

```
    if ch == flag or ch == escape:
```

```
        stuffed += escape
```

```
    stuffed += ch
```

```
stuffed += flag
```

```
print("After Byte Stuffing:", stuffed)
```

Output:

```
Enter data: ABCFLAGDESCAPE
```

```
After Byte Stuffing: FLAGABCFLAGDESCAPEFLAG
```

5. Write a program to implement stop and wait protocol.**Program:**

Write a Program to implement stop and wait protocol.

```
import random
```

```
import time
```

```
frames = random.randint(1,20)
```

```
print("Number of Frames is",frames)
```

```
for i in range(1,frames+1):
```

```
    print("Sending Frame",i)
```

```
    x=random.randint(0,9)
```

```
    if x%2==0:
```

```
        print("Waiting for 1 Second")
```

```
        time.sleep(1)
```

```
        print("Sending Frame",i)
```

```
    print("ACK for Frame",i)
```

```
print("End of Stop and Wait Protocol")
```

Output:

```
Number of Frames is 4
Sending Frame 1
Waiting for 1 Second
Sending Frame 1
ACK for Frame 1
Sending Frame 2
Waiting for 1 Second
Sending Frame 2
ACK for Frame 2
Sending Frame 3
Waiting for 1 Second
Sending Frame 3
ACK for Frame 3
Sending Frame 4
Waiting for 1 Second
Sending Frame 4
ACK for Frame 4
End of Stop and Wait Protocol
```

6. Write a program to implement Go Back N protocol.**Program:**

```
window = int(input("Enter Window Size: "))
```

```
total = int(input("Enter Total Frames: "))
```

```
sent = 0
```

```
while sent < total:
```

```
    for i in range(window):
```

```
        if sent < total:
```

```
            print("Frame", sent, "has been transmitted.")
```

```
            sent += 1
```

```
ack = int(input("Enter last Acknowledgement received: "))
```

```
if ack >= sent - 1:
```

```
    print("Acknowledgement received.")
```

```
else:
```

```
    print("Resending from Frame", ack)
```

```
    sent = ack
```

```
print("All frames sent successfully")
```

Output:

```
Enter Window Size: 3
Enter Total Frames: 6
Frame 0 has been transmitted.
Frame 1 has been transmitted.
Frame 2 has been transmitted.
Enter last Acknowledgement received: 2
Acknowledgement received.
Frame 3 has been transmitted.
Frame 4 has been transmitted.
Frame 5 has been transmitted.
Enter last Acknowledgement received: 5
Acknowledgement received.
All frames sent successfully
```

7. Write a program to implement Selective Repeat Protocol.**Program:**

```
n = int(input("Enter number of frames: "))
```

```
w = int(input("Enter window size: "))
```

```
i = 0
```

```
while i < n:
```

```
    for j in range(i, min(i + w, n)):
```

```
        print("Sending Frame", j)
```

```
    lost = int(input("Enter lost frame (-1 if none): "))
```

```
    if lost == -1:
```

```
        i += w
```

```
    else:
```

```
        print("Resending Frame", lost)
```

```
    i = lost + 1
```

```
print("All frames sent successfully")
```

Output:

```
Enter number of frames: 5
Enter window size: 3
Sending Frame 0
Sending Frame 1
Sending Frame 2
Enter lost frame (-1 if none): 1
Resending Frame 1
Sending Frame 2
Sending Frame 3
Sending Frame 4
Enter lost frame (-1 if none): 4
Resending Frame 4
All frames sent successfully
```

8. Write a program to implement Distance Vector Routing Algorithm.**Program:**

```
# Distance Vector Routing Algorithm
```

```
INF = 999
```

```
n = int(input("Enter number of routers: "))
```

```
print("Enter the cost matrix:")
```

```
cost = []
```

```
for i in range(n):
```

```
    cost.append(list(map(int, input().split())))
```

```
# Initially, distance table = cost matrix
```

```
dist = [row[:] for row in cost]
```

```
# Distance Vector Algorithm
```

```
for k in range(n):
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            if dist[i][j] > dist[i][k] + dist[k][j]:
```

```
                dist[i][j] = dist[i][k] + dist[k][j]
```

```
# Display shortest distances
```

```
print("\nShortest Distance Table:")
```

```
for i in range(n):  
    print("Router", i + 1, ":", dist[i])
```

Output:

```
Enter number of routers: 4  
Enter the cost matrix:  
0 2 5 999  
2 0 1 3  
5 1 0 2  
999 3 2 0
```

```
Shortest Distance Table:  
Router 1 : [0, 2, 3, 5]  
Router 2 : [2, 0, 1, 3]  
Router 3 : [3, 1, 0, 2]  
Router 4 : [5, 3, 2, 0]
```

9. Write a program to implement Dijkstra's Algorithm.**Program:**

```
# Dijkstra's Algorithm
```

```
INF = 999
```

```
n = int(input("Enter number of vertices: "))
```

```
print("Enter the cost matrix:")
```

```
graph = []
```

```
for i in range(n):
```

```
    graph.append(list(map(int, input().split())))
```

```
source = int(input("Enter source vertex: ")) - 1
```

```
distance = [INF] * n
```

```
visited = [False] * n
```

```
distance[source] = 0
```

```
for i in range(n):
```

```
    # Find nearest unvisited vertex
```

```
    min_dist = INF
```

```
    u = -1
```

```
    for j in range(n):
```

```
    if not visited[j] and distance[j] < min_dist:
        min_dist = distance[j]
        u = j

    if u == -1:
        break

    visited[u] = True

    # Update distances
    for v in range(n):
        if graph[u][v] != 0 and not visited[v]:
            new_dist = distance[u] + graph[u][v]

            if new_dist < distance[v]:
                distance[v] = new_dist

    print("\nShortest distances from vertex", source + 1)

    for i in range(n):
        print("To vertex", i + 1, "=", distance[i])
```

Output:

```
Enter number of vertices: 4
```

```
Enter the cost matrix:
```

```
0 2 5 0
```

```
2 0 1 3
```

```
5 1 0 2
```

```
0 3 2 0
```

```
Enter source vertex: 1
```

```
Shortest distances from vertex 1
```

```
To vertex 1 = 0
```

```
To vertex 2 = 2
```

```
To vertex 3 = 3
```

```
To vertex 4 = 5
```

10. Write a program to implement echo Server and Client using TCP socket.**Server Program:**

Write a program to implement echo Server using TCP socket.

```
import socket
```

```
server = socket.socket()
```

```
server.bind(("localhost", 5000))
```

```
server.listen(1)
```

```
print("Server is running...")
```

```
conn, addr = server.accept()
```

```
print("Client connected")
```

```
while True:
```

```
    msg = conn.recv(1024).decode()
```

```
    if msg == "exit":
```

```
        break
```

```
    print("Received:", msg)
```

```
    conn.send(msg.encode())
```

```
conn.close()
```

```
server.close()
```

```
print("Server closed")
```

Output:

```
Server is running...
Client connected
Received: This is Network Lab Assignment.
Received: Hi
Server closed
```

Client Program:

Write a program to implement echo client using TCP socket.

```
import socket
```

```
client = socket.socket()
```

```
client.connect(("localhost", 5000))
```

```
while True:
```

```
    msg = input("Enter message: ")
```

```
    client.send(msg.encode())
```

```
    if msg == "exit":
```

```
        break
```

```
    reply = client.recv(1024).decode()
```

```
    print("Echo from server:", reply)
```

```
client.close()
```

```
print("Client closed")
```

Output:

```
$ python "TCP Client.py"  
Enter message: This is Network Lab Assignment.  
Echo from server: This is Network Lab Assignment.  
Enter message: Hi  
Echo from server: Hi  
Enter message: exit  
Client closed
```

11. Write a program to implement echo Server and Client using UDP socket.**Server Program:**

Write a program to implement echo server using UDP socket.

```
import socket
```

```
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

```
server.bind(("localhost", 5000))
```

```
print("UDP Echo Server is running...")
```

```
while True:
```

```
    data, address = server.recvfrom(1024)
```

```
    print("Client:", data.decode())
```

```
    server.sendto(data, address)
```

Output:

```
UDP Echo Server is running...
Client: Hi Everyone
Client: How are you?
```

Client Program:

Write a program to implement echo client using UDP socket.

```
import socket
```

```
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

```
while True:
```

```
    msg = input("Enter message: ")
```

```
    if msg == "exit":
```

```
        break
```

```
    client.sendto(msg.encode(), ("localhost", 5000))
```

```
    data, address = client.recvfrom(1024)
```

```
    print("Echo from server:", data.decode())
```

```
client.close()
```

Output:

```
$ python "UDP Client.py"
Enter message: Hi Everyone
Echo from server: Hi Everyone
Enter message: How are you?
Echo from server: How are you?
Enter message: exit
```

12. Write a socket for HTTP for web page uploads and downloads.**Server Program:**

```
import socket
import os

server = socket.socket()
server.bind(("localhost", 8000))
server.listen(1)

print("Server is running...")

while True:
    conn, addr = server.accept()

    request = conn.recv(4096).decode(errors="ignore")
    print(request)

    if request.startswith("GET"):
        filename = "file.txt"

        if os.path.exists(filename):
            with open(filename, "rb") as f:
                data = f.read()

            response = b"HTTP/1.1 200 OK\r\n\r\n" + data
        else:
            response = b"HTTP/1.1 404 Not Found\r\n\r\nFile not found"
```

```
conn.sendall(response)
```

```
conn.close()
```

Output:

```
Server is running...  
GET /file.txt HTTP/1.1  
Host: localhost
```

Client Program:

```
import socket
```

```
client = socket.socket()
```

```
client.connect(("localhost", 8000))
```

```
request = "GET /file.txt HTTP/1.1\r\nHost: localhost\r\n\r\n"
```

```
client.send(request.encode())
```

```
data = client.recv(4096)
```

```
print(data.decode(errors="ignore"))
```

```
client.close()
```

Output:

```
$ python "HTTP Client.py"  
HTTP/1.1 200 OK
```

```
Hi, This is My HTTP file.
```

13. Write a program to implement RPC (Remote Procedure call).**Server Program:**

```
# RPC Server
```

```
from xmlrpc.server import SimpleXMLRPCServer
```

```
def add(a, b):
```

```
    return a + b
```

```
server = SimpleXMLRPCServer(("localhost", 8000))
```

```
server.register_function(add, "add")
```

```
print("RPC Server is running...")
```

```
server.serve_forever()
```

Output:

```
> RPC Server is running...  
127.0.0.1 - - [04/Oct/2026 16:28:08] "POST /RPC2 HTTP/1  
.1" 200 -
```

Client Program:

```
import socket
```

```
client = socket.socket()
```

```
client.connect(("localhost", 8000))
```

```
request = "GET /file.txt HTTP/1.1\r\nHost: localhost\r\n\r\n"
```

```
client.send(request.encode())
```

```
data = client.recv(4096)
```

```
print(data.decode(errors="ignore"))
```

```
client.close()
```

Output:

```
$ python "RPC Client.py"  
Enter first number: 3  
Enter second number: 4  
Result = 7
```